МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования
«Гомельский государственный технический университет имени П.О. Сухого»

Факультет автоматизированных и информационных систем

Кафедра «Информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине: «ООПиП»
на тему: «Потоки ввода/вывода и порождающие паттерны: Фабричный метод и Абстрактная фабрика.»

Выполнил: студент гр. ИТП-21
Лосев М.А.
Принял: ассистент
Карась О.В.

Гомель 2026

**Цель работы:** Освоить механизмы сохранения и восстановления состояния объектов (сериализация/десериализация) с использованием потоков ввода/вывода и формата *XML*. Научиться применять порождающие паттерны для динамического создания объектов на основе конфигурационных данных.

**Задание:**

Датчики контроля: Температуры, Давления, Влажности.

Описание: *XML* содержит схему размещения датчиков на участке. Фабрика восстанавливает объекты с их пороговыми значениями.

## 1. Описание программы

Разработанное приложение представляет собой систему управления датчиками, реализованную на платформе *.NET* 10 с использованием *WinForms*. Программа разделена на два проекта: библиотеку классов (*ClassLib*) с бизнес-логикой и основное приложение.

Основные возможности:

1. Иерархия объектов: Реализованы классы для температурных, гигрометрических датчиков и датчиков давления.
2. Паттерн «Фабрика»: Создание объектов происходит динамически через фабричный метод на основе перечисления (*enum*).
3. Два режима парсинга: Реализована обработка *XML* через *DOM* (*XmlDocument*) для удобной навигации и *SAX* (*XmlReader*) для быстрого потокового чтения.
4. Валидация: Данные проверяются на соответствие схеме *XSD* перед загрузкой.
5. Фильтрация: Используется технология *LINQ to XML* для поиска датчиков, превышающих заданный порог.

## 2. Графический интерфейс *WinForms*:

1. Левая колонка – фильтры и действия.
2. Правая колонка – результаты фильтров и группировок.

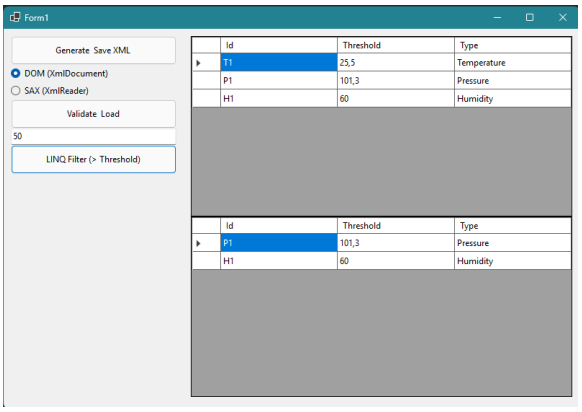

Рисунок 1 – Интерфейс программы

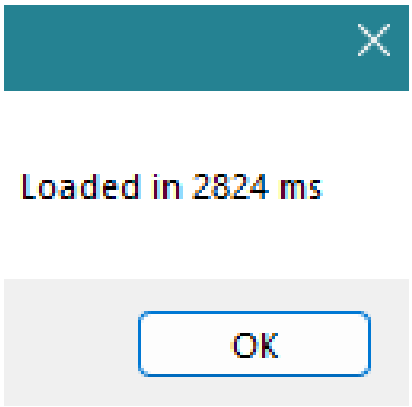


Рисунок 2 – Результат загрузки данных

### 3. *UML* – диаграмма классов системы

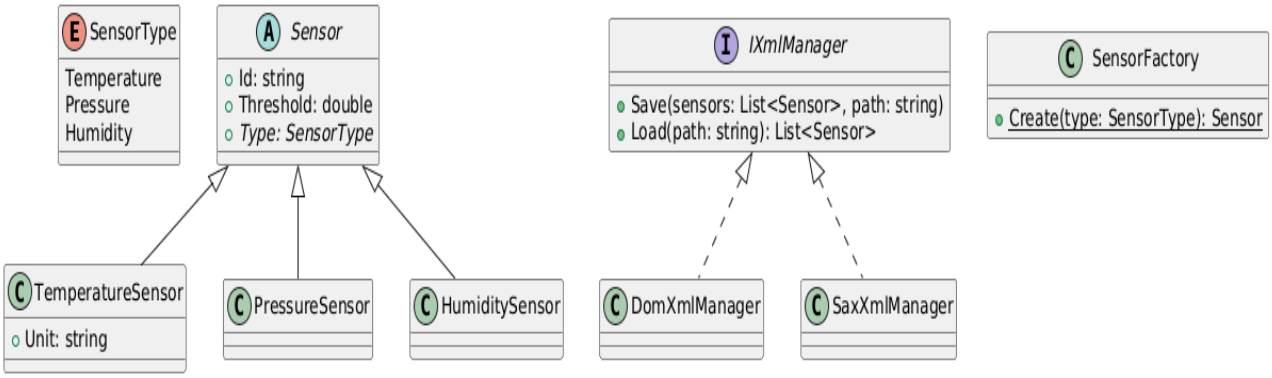


Рисунок 3 – *UML* – диаграмма классов системы

**Вывод:** В ходе лабораторной работы были изучены и практически применены методы работы с *XML*-документами в среде *.NET*. Реализация двух различных подходов к парсингу (*DOM* и *SAX*) позволила сравнить их эффективность и ресурсозатратность. Использование паттерна «Фабричный метод» обеспечило гибкую архитектуру приложения, позволяющую легко расширять список типов датчиков без изменения логики загрузки данных. Закреплены навыки разделения ответственности между логикой (*ClassLib*) и интерфейсом (*WinForms*), а также механизмы валидации данных через *XSD*-схемы.

# ПРИЛОЖЕНИЕ А
(обязательное)

*MainWindow.xaml.cs*

```
using System.Windows;
using System.Text.Json;
using Second.ClassLib.First;
using Second.Wpf.First.StageManagers;

namespace Second.Wpf.First;

public partial class MainWindow : Window
{
    private readonly MyLinkedList<AssemblyStage> data = new
MyLinkedList<AssemblyStage>();
    private readonly IStageManager manager = new StageManagerQueryable();

    public MainWindow()
    {
        InitializeComponent();

        SeedData();
        RefreshSourceView();
        RefreshResultView(data);
    }

    private void SeedData()
    {
        data.Add(new ManualStage("Cutting", 40, DateTime.Now, "Workshop1",
DangerLevel.High, true));
        data.Add(new AutomatedStage("Welding", 25, DateTime.Now, "Workshop2",
DangerLevel.Medium, "M-01"));
        data.Add(new ManualStage("Painting", 55, DateTime.Now, "Workshop1",
DangerLevel.Low, false));
        data.Add(new AutomatedStage("Inspection", 15, DateTime.Now, "Workshop3",
DangerLevel.Low, "M-02"));
    }

    private void RefreshSourceView()
    {
        SourceList.ItemsSource = data.Select(ToJson).ToList();
    }

    private void RefreshResultView(IEnumerable<AssemblyStage> result)
    {
        ResultList.ItemsSource = result.Select(ToJson).ToList();
    }

    private static string ToJson(AssemblyStage stage)
    {
        return JsonSerializer.Serialize(stage, new JsonSerializerOptions
{ WriteIndented = false });
    }

    private void Apply_Click(object sender, RoutedEventArgs e)
    {
        var query = data.AsEnumerable();
```

```
        if (UseDurationFilter.IsChecked == true && int.TryParse(DurationBox.Text,
out int minDuration))
            query = manager.GetStagesLongerThan(query, minDuration);

        if (UseWorkshopFilter.IsChecked == true && !
string.IsNullOrWhiteSpace(WorkshopBox.Text))
            query = query.Where(s => s.Workshop == WorkshopBox.Text);

        RefreshResultView(query);
    }

    private void Reset_Click(object sender, RoutedEventArgs e)
    {
        UseDurationFilter.IsChecked = true;
        UseWorkshopFilter.IsChecked = false;
        DurationBox.Text = "30";
        WorkshopBox.Text = "";
        AverageResult.Text = "";
        GroupResult.ItemsSource = null;

        RefreshResultView(data);
    }

    private void Average_Click(object sender, RoutedEventArgs e)
    {
        if (string.IsNullOrWhiteSpace(WorkshopBox.Text))
        {
            MessageBox.Show("Enter a workshop to calculate average");
            return;
        }

        double avg = manager.GetAverageDurationByWorkshop(data, WorkshopBox.Text);
        AverageResult.Text = $"Average duration in {WorkshopBox.Text}: {avg:F2}
min";
    }

    private void Group_Click(object sender, RoutedEventArgs e)
    {
        var groups = manager.GroupByDangerLevel(data);

        GroupResult.ItemsSource = groups
            .Select(g => $"{g.Key}: {g.Count()} stage(s)")
            .ToList();
    }
}
```